

Escuela Técnica Superior de Ingeniería

Universidad de Huelva

Grado en Ingeniería Informática

Inteligencia Artificial

Práctica 1

Luis Airam Saavedra Paiseo

Marzo 2026

Índice

Índice.....	2
1. Introducción.....	3
2. Descripción del Problema	3
3. Agentes	4
3.1 Agente Interactivo	4
Bugs encontrados y soluciones.....	4
3.2 Agente Reactivo	5
Bugs encontrados y soluciones.....	5
3.3 El Agente que lo sabía todo (Greedy / Deliberativo 2C)	6
Bugs encontrados y soluciones.....	6
3.4 El Agente que podía ver el futuro (Deliberativo 2A).....	7
Bugs encontrados y soluciones.....	7
3.5 El Agente que se acordaba de todo (Deliberativo 2B)	8
Bugs encontrados y soluciones.....	8
4. Conclusiones	11

1. Introducción

Esta práctica está basada en el juego Tron, consistido en navegar a través de un mapa, evitando chocar con muros y tu propia estela. Esta práctica ha hecho una adaptación a un laberinto, dejando una estela por donde pase el Agente pasando a ser un nuevo obstáculo, haciendo que la toma de decisiones deba considerarlo, así como deberá considerar las consecuencias acarreadas por dichas acciones.

2. Descripción del Problema

El proyecto consiste en 2 mapas de caracteres, uno de tamaño pequeño con una "trampa" en forma de callejón, y otro más grande con más posibilidades de rutas que lleven a la salida. El objetivo de los Agentes es conseguir llegar desde el punto de inicio A hasta la salida S, evitando las paredes # y las estelas que irán dejando allá por donde pasen.

```
static final String MAPA_CALLEJON =
    "#####\n" +
    "#A      #\n" +
    "# ##### #\n" +
    "# #   # #\n" +
    "#     S #\n" +
    "#####";
```

Mapa 1: Mapa pequeño con callejón trampa al Este desde la posición inicial.

```
static final String MAPA_GRANDE =
    "#####\n" +
    "#A  #                #\n" +
    "# # # ##### #\n" +
    "# #   #   #   #\n" +
    "# ##### #\n" +
    "#     #   #   #\n" +
    "# ### ##### # #\n" +
    "#  #           #\n" +
    "# ##### S\n" +
    "#####";
```

Mapa 2: Mapa grande con múltiples rutas y mayor complejidad.

El proyecto base gira en torno a 2 métodos que proporcionan la lógica del resto, puesto que no son los triviales y son los que habilitan los movimientos del agente. Además de eso se nos proporciona un esqueleto con las pruebas necesarias para solo tener que implementar los propios agentes.

```
public boolean esTransitable(int f, int c) {
    if (f < 0 || f >= filas || c < 0 || c >= cols) return false;
    return grid[f][c] != '#' && grid[f][c] != '.';
}
```

Código 1: Método que permite saber si una casilla con coordenada fila f y columna c es transitable o no.

```
public char verCasilla(int f, int c) {
    if (f < 0 || f >= filas || c < 0 || c >= cols) return '#';
    return grid[f][c];
}
```

3. Agentes

Todos los Agentes partirán de una clase padre llamada Agente, la cual tiene la cabecera del método abstracto pensar(), que deberán implementar todos utilizándolo de la manera que convenga según el agente, y el método percibir(), que observará en las 4 direcciones posibles desde la casilla pasada por parámetro y devolverá un array de 4 posiciones de booleanos correspondientes a las 4 direcciones (Norte, Sur, Este y Oeste).

```
public abstract class Agente {
    protected Random generador = new Random();
    public abstract String pensar(Entorno mapa);
    static boolean[] percibir(int f, int c, Entorno mapa) {
        boolean acciones[] = new boolean[4];
        if(mapa.esTransitable(f-1, c)) acciones[0] = true; // N
        if(mapa.esTransitable(f+1, c)) acciones[1] = true; // S
        if(mapa.esTransitable(f, c+1)) acciones[2] = true; // E
        if(mapa.esTransitable(f, c-1)) acciones[3] = true; // O
        return acciones;
    }
}
```

Código 3: Implementación de la clase padre Agente con el método percibir() compartido.

3.1 Agente Interactivo

El primer Agente a implementar, trivial y determinista en cierta parte, pues se mueve en base a la entrada que recibe del usuario, siguiendo siempre el mismo camino si así lo eligiese. Este Agente es mayormente usado como familiarización con el entorno, además de totalmente dependiente de las decisiones del usuario para encontrar la meta.

Bugs encontrados y soluciones

Durante el desarrollo se encontró el siguiente problema:

Bug: fila y col no se reseteaban en cada iteración

Si el usuario introducía una acción inválida, el bucle pedía otra. Pero si ésta también era inválida el agente se estrellaba, ya que las variables fila y col se modificaban acumulativamente sin resetearse al inicio de cada iteración.

```
int fila = entorno.agenteF; // inicializadas FUERA del bucle
int col = entorno.agenteC; // solo se calculan una vez
do {
    accion = scanner.nextLine();
    if(accion.equals("N")) { fila--; } // si invalido, fila queda en fila-1
    // siguiente iteracion parte de fila-1, no de la posicion real
} while(!entorno.esTransitable(fila, col));
```

Solución: mover la inicialización de fila y col al inicio del bucle

```
int fila, col; // solo se declaran aqui
do {
```

```

    fila = entorno.agenteF;    // reset en cada iteracion
    col  = entorno.agenteC;    // siempre parte de la posicion real
    accion = scanner.nextLine();
    if(accion.equals("N")) { fila--; }
} while(!entorno.esTransitable(fila, col));

```

3.2 Agente Reactivo

Primer Agente con una forma de razonar detrás, percibe su entorno y posteriormente a ello, elige aleatoriamente 1 dirección entre todas las posibles como acciones inmediatas. En caso de quedarse atascado, se ha optado por elegir una acción estándar a devolver (Norte), de forma que se fuerce el choque y no haya que consumir el resto de ciclos para finalizar la ejecución.

Este Agente es capaz de resolver ambos mapas, pero se desenvuelve mejor en mapas sin muchos obstáculos, ya que tiende a entrar en bucles repetitivos y quedarse encerrado.

Bugs encontrados y soluciones

● Bug 1: AgenteReactivo definida como clase interna de AgenteInteractivo

En Java las clases internas no estáticas necesitan una instancia de la clase exterior para instanciarse, lo que impedía crear AgenteReactivo directamente desde el main.

Solución: sacar AgenteReactivo fuera como clase independiente al mismo nivel que AgenteInteractivo.

● Bug 2: Array de acciones con tamaño 0

El array se inicializaba con el valor de n en ese momento (0), por lo que tenía longitud 0 y fallaba al intentar escribir en él.

```

int n = 0;
String[] acciones = new String[n];    // tamaño 0, falla al escribir
if(mapa.esTransitable(fila-1, col)) {
    acciones[n] = "N";    // ArrayIndexOutOfBoundsException
    n++;
}

```

✓ Solución: inicializar el array con tamaño fijo 4 (máximo posible de direcciones)

```

int n = 0;
String[] acciones = new String[4];    // tamaño fijo 4
if(mapa.esTransitable(fila-1, col)) {
    acciones[n] = "N";    // correcto
    n++;
}

```

● Bug 3: else if en lugar de if independientes + operadores ++ en lugar de +1/-1

Con else if solo se evaluaba la siguiente dirección si la anterior no era transitable, impidiendo comparar todas las opciones. Además, fila++ y col++ modificaban la variable permanentemente en lugar de calcular la posición destino.

```

if(mapa.esTransitable(fila++, col)) { acciones[n]="N"; n++; }
else if(mapa.esTransitable(fila--, col)) { acciones[n]="S"; n++; }
else if(mapa.esTransitable(fila, col++)) { acciones[n]="E"; n++; }

```

```
// Solo evalua la siguiente si la anterior NO es transitable
// fila++ modifica fila permanentemente
```

✓ **Solución: if independientes con desplazamientos +1/-1 sin modificar la variable**

```
if(mapa.esTransitable(fila-1, col)) { acciones[n]="N"; n++; }
if(mapa.esTransitable(fila+1, col)) { acciones[n]="S"; n++; }
if(mapa.esTransitable(fila, col+1)) { acciones[n]="E"; n++; }
if(mapa.esTransitable(fila, col-1)) { acciones[n]="O"; n++; }
// if independientes + coordenadas sin modificar la variable
```

● **Bug 4: Random creado dentro del método pensar()**

Al instanciar Random en cada llamada con semilla de milisegundos, la semilla era frecuentemente la misma entre ciclos rápidos (300ms de pausa) y el agente siempre elegía la misma acción.

```
public String pensar(Entorno mapa) {
    // se crea nuevo Random en cada ciclo con misma semilla
    Random generador = new Random(System.currentTimeMillis());
    int indice = generador.nextInt(acciones.length);
    return acciones[indice]; // siempre el mismo resultado
}
```

✓ **Solución: Random como atributo heredado de la clase Agente, se crea una sola vez**

```
class AgenteReactivo extends Agente {
    // atributo de clase, se crea UNA sola vez
    // heredado de Agente como generador
    public String pensar(Entorno mapa) {
        int indice = generador.nextInt(n);
        return acciones[indice];
    }
}
```

3.3 El Agente que lo sabía todo (Greedy / Deliberativo 2C)

Este Agente tendrá conocimiento de la posición de la meta con sus coordenadas fila f y columna c. Se ha optado por la estrategia de elegir la dirección con la menor distancia Manhattan, calculada con la fórmula matemática:

$$|x_1 - x_2| + |y_1 - y_2|$$

En 2 pasadas se obtiene la menor distancia Manhattan de entre todas las transitables y en caso de empate se elige de forma aleatoria el siguiente movimiento. Este Agente resuelve ambos mapas con facilidad.

Bugs encontrados y soluciones

● **Bug 1: Fórmula Manhattan con un solo Math.abs()**

Todo dentro de un solo Math.abs() hace que las diferencias de fila y columna puedan cancelarse entre sí, dando resultados incorrectos.

```
// Un solo Math.abs(): diferencias se cancelan entre si
// Ejemplo: si df=+3 y dc=-3, da 0 cuando debería dar 6
distancia = Math.abs(fila-1 - mapa.metaF + col - mapa.metaC);
```

✓ **Solución: dos Math.abs() separados, uno para filas y otro para columnas**

```
// Dos Math.abs() separados, uno para filas y otro para columnas
distancia = Math.abs(fila-1 - mapa.metaF) + Math.abs(col - mapa.metaC);
```

● Bug 2: Llaves olvidadas en el segundo for

Sin llaves, n++ se ejecutaba siempre independientemente del if, acumulando índices aunque no se hubiera añadido ninguna acción. Esto dejaba posiciones null en el array que causaban NullPointerException al intentar usarlas.

```
for(int i = 0; i < 4; i++) {
    if(posibles[i])
        distancia = distManhatan(fila+df[i], col+dc[i], mapa);
        if(distancia == menor) { accion[n] = nombres[i]; n++; }
        // n++ se ejecuta SIEMPRE aunque no cumpla la condicion
        // deja posiciones null en el array -> NullPointerException
}
```

✓ Solución: llaves correctas para que n++ solo se ejecute dentro del if

```
for(int i = 0; i < 4; i++) {
    if(posibles[i]) {
        distancia = distManhatan(fila+df[i], col+dc[i], mapa);
        if(distancia == menor) { accion[n] = nombres[i]; n++; }
    } // llaves correctas: n++ solo dentro del if
}
```

3.4 El Agente que podía ver el futuro (Deliberativo 2A)

Este Agente tiene la capacidad de ver más allá de los 4 movimientos iniciales, llegando a evaluar 16 posibles movimientos: N, S, E, O, NN, NE, NO, SS, SE, SO, EE, EN, ES, OO, ON, OS. Implementa 2 estrategias en consonancia:

1. Si alguna de las 16 combinaciones lleva directamente a la salida S en 1 o 2 pasos, devuelve esa acción inmediatamente.
2. Si no hay salida cercana, calcula el número de huecos disponibles desde cada casilla destino y elige la dirección con más huecos. Desempate aleatorio.

Este Agente tiene un comportamiento más determinista y funciona con éxito en ambos laberintos.

Bugs encontrados y soluciones

● Bug 1: Coordenadas a 2 pasos mal calculadas

Para NN se usaba fila-1 en lugar de fila-2, lo que comprobaba Norte dos veces en lugar de dos pasos al Norte.

```
// Para NN se comprobaba fila-1 en lugar de fila-2
if(mapa.esTransitable(fila-1, col)) { // esto es N, no NN
    if(fila-1 == mapa.metaF && col == mapa.metaC) {
        return nombres[i];
    }
}
```

✓ Solución: fila-2 para NN, fila+2 para SS, col+2 para EE, col-2 para OO

```
// NN requiere fila-2
if(mapa.esTransitable(fila-2, col)) { // NN correcto
```

```

    if(fila-2 == mapa.metaF && col == mapa.metaC) {
        return nombres[i];
    }
}

```

● Bug 2: Array opuestos incorrecto y condición invertida

El array de opuestos tenía {0,1,3,2} en lugar de {1,0,3,2}, y la condición usaba == en lugar de != para excluir el retroceso.

```

int opuestos[] = {0, 1, 3, 2}; // MAL: Norte(0) opuesto es Norte(0)
// ...
if(j == opuestos[i]) { // MAL: evalúa SOLO el opuesto
    // debería ser != para evaluar TODOS excepto el opuesto
}

```

✓ Solución: opuestos correctos y != para evaluar todas las direcciones excepto la opuesta

```

int opuestos[] = {1, 0, 3, 2}; // Norte(0)<->Sur(1), Este(2)<->Oeste(3)
// ...
if(j != opuestos[i]) { // evalúa todas EXCEPTO la opuesta
    if(posibles2[j]) {
        // comprobar si lleva a la salida
    }
}

```

● Bug 3: nc usaba df en lugar de dc

Al calcular la columna del destino a 2 pasos, se usaba el array de deltas de fila (df) en lugar del de columna (dc).

```

// nc usaba df en lugar de dc
int nf = fila + df[i] + df[j];
int nc = col + df[i] + df[j]; // MAL: debería usar dc

```

✓ Solución: usar dc para columnas y df para filas

```

int nf = fila + df[i] + df[j];
int nc = col + dc[i] + dc[j]; // correcto: dc para columnas

```

3.5 El Agente que se acordaba de todo (Deliberativo 2B)

Este Agente hace uso de una memoria interna, rellenándola progresivamente con lo que va percibiendo del mapa real. La memoria distingue entre casillas pisadas ('.'), vistas ('v'), muros ('#') y desconocidas ('\0').

Se han implementado 2 estrategias en orden de prioridad:

1. Inercia pura: continuar en la misma dirección siempre que sea transitable. Solo cambia al toparse con un muro o su propia estela.
2. numPasos sobre memoria interna: si la inercia no aplica, cuenta pasos disponibles en casillas conocidas y libres ('v') de cada dirección. Elige la de más pasos. Desempate aleatorio.

Gracias a estas 2 estrategias en consonancia, el Agente resuelve ambos laberintos con muy poca componente aleatoria.

Bugs encontrados y soluciones

● Bug 1: ultimaAccion como variable local

Al declararse dentro del método pensar(), se reseteaba a "" en cada ciclo y la inercia nunca podía funcionar.

```
public String pensar(Entorno mapa) {  
    String ultimaAccion = ""; // variable LOCAL del metodo  
    // se resetea a "" en cada ciclo  
    // la inercia nunca puede funcionar  
}
```

✓ Solución: moverla como atributo de clase para que persista entre ciclos

```
class AgenteDeliberativo2B extends Agente {  
    String ultimaAccion = ""; // atributo de CLASE  
    // persiste entre ciclos, la inercia funciona correctamente  
}
```

● Bug 2: Comparar Strings con == en lugar de .equals()

En Java, == compara referencias de memoria, no el contenido del String. Esto hace que la comparación falle silenciosamente en muchos casos.

```
if(ultimaAccion != "") { // == y != comparan referencias en Java  
    if(ultimaAccion == "N") { ... } // puede fallar silenciosamente  
}
```

✓ Solución: usar .equals() para comparar el contenido de los Strings

```
if(!ultimaAccion.equals("")) { // .equals() compara contenido  
    if(ultimaAccion.equals("N")) { ... }  
}
```

● Bug 3: Bloque de fallback dentro de else

El bloque de evaluación de las 4 direcciones estaba dentro de un else, ejecutándose únicamente cuando ultimaAccion era "", es decir, solo en el primer ciclo. A partir del segundo ciclo, si la inercia fallaba no había ningún fallback.

```
// Bloque de fallback dentro de else: solo ejecuta en el primer ciclo  
if(!ultimaAccion.equals("")) {  
    // comprobacion inercia...  
}  
else { // solo se ejecuta si ultimaAccion==""  
    if(mapa.esTransitable(fila-1, col)) { ... } // nunca llega aqui  
}
```

✓ Solución: eliminar el else para que el fallback sea siempre accesible

```
// Sin else: el fallback siempre es accesible cuando la inercia falla  
if(!ultimaAccion.equals("")) {  
    // si la inercia se cumple -> return directo  
    // si no se cumple -> cae al bloque de abajo  
}  
// fallback siempre disponible  
for(int i = 0; i < 4; i++) {  
    if(posibles[i]) { acciones[n] = nombres[i]; n++; }  
}
```

● Bug 4: numPasos empezaba en i=0

Con i=0 la primera comprobación era la posición actual del agente, que ya está marcada como '.' en memoria y siempre rompía el bucle inmediatamente dando 0 pasos.

```
// Empieza en i=0: la posicion actual ya tiene '.' en memoria
// siempre daría 0 pasos en la primera iteracion
for(int i = 0; ; i++) {
    char casilla = mem[f + i*df][c + i*dc];
    if(casilla == '\0' || ...) break;
    pasos++;
}
```

✓ Solución: empezar en i=1 para saltar la posición actual y evaluar las siguientes

```
// Empieza en i=1: salta la posicion actual y evalua las siguientes
for(int i = 1; ; i++) {
    int nf = f + i*df, nc = c + i*dc;
    if(nf<0 || nf>=mapa.filas || nc<0 || nc>=mapa.cols) break;
    char casilla = mem[nf][nc];
    if(casilla == '\0' || casilla == '.' || casilla == '#') break;
    pasos++;
}
```

4. Conclusiones

Si tuviésemos que comparar a todos los Agentes implementados en esta práctica, nos daríamos cuenta como, a medida que avanzamos, los Agentes cada vez son más y más deterministas y menos triviales. Por ejemplo, el Reactivo y el Deliberativo 2B en las mismas condiciones iniciales: el resultado más probable será que el Deliberativo 2B resuelva el laberinto en una mayor cantidad de casos, o que lo resuelva de manera más eficiente.

Obviamente, todos fallan en algún punto, ya que todos poseen componente de aleatoriedad en mayor o menor medida, haciendo que no sean capaces de siempre resolver el laberinto propuesto.

El punto más importante a destacar es el comportamiento de los Agentes Deliberativos frente a los Reactivos. Los Deliberativos, antes de actuar, utilizan la información recopilada del proceso de percepción a través de los sensores de los que dispongan para utilizarlos en la toma de decisiones de manera coherente y siguiendo una estrategia. Mientras que los Reactivos perciben la información pero no tienen porqué utilizarla para la toma de decisiones, obteniendo así peores resultados.

Gracias a la capacidad de poder incorporar estrategias y estrategias auxiliares, hemos podido observar una notoria mejora en cuanto a los resultados obtenidos con los Agentes 2A y 2B sobre todo, que son los más restrictivos y deterministas de entre todos los implementados.